CSCI 5302 HW2

February 11, 2026

Ram Prasad

Q1

The trajectory of the water stream obeys

$$y = x\left[\frac{g}{rv\cos(\alpha)} + \tan(\alpha)\right] + \frac{g}{r^2}\log\left[1 - \frac{rx}{v\cos(\alpha)}\right]$$

where for a fixed horizontal distance x, air resistance r, water velocity v, and gravitational constant g given, the vertical distance y is a function of angle α. The solution to

$$y(\alpha) - y_{\text{target}} = 0$$

is the α which hits the target at horizontal distance x_{target} and height equal to y_{target}. Let $g(\alpha) = y(\alpha) - y_{\text{target}}$. Computing $g'(\alpha)$ gives

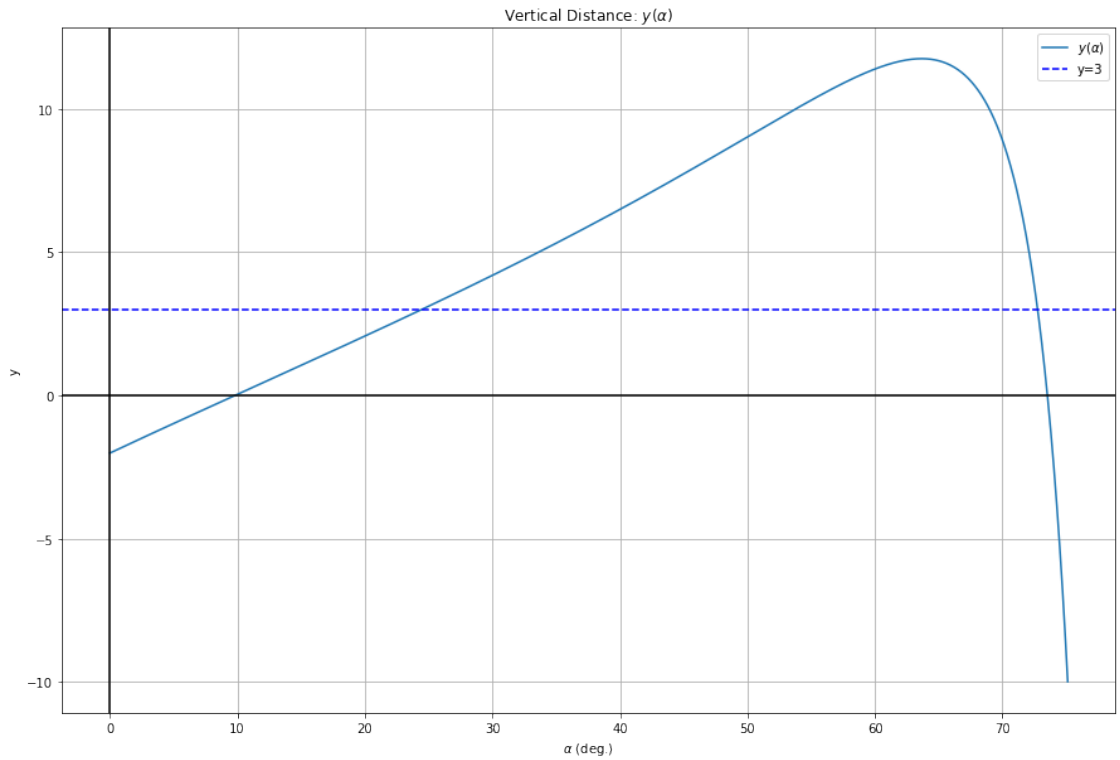
$$g'(\alpha) = x\left[\frac{g\sin(\alpha)}{rv\cos^2(\alpha)} + \sec^2(\alpha)\right] - \frac{g}{r^2}\cdot\frac{1}{1-\frac{rx}{v\cos(\alpha)}}\cdot\frac{rx\sin(\alpha)}{v\cos^2(\alpha)}$$

[1]:
```
# standard imports
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import math
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
# define trajectory function
def trajectory(a,x=12,g=9.8065,r=0.30,v=20,y_target=0):
    """
    Function for trajectory of water stream.
    Args:
    -----
        x (float): horizontal distance
        g (float): acceleration of gravity
```

```
        r (float): air resistance
        v (float): initial water velocity
        a (float): angle in degrees
    Returns:
    --------
        (float): function value
    """
    # convert degrees to radians
    a = a*np.pi/180.0

    # return function value
    return x*( g/(r*v*np.cos(a)) + np.tan(a)) + (g/(r**2))*np.log( 1 - (r*x)/
 ↪(v*np.cos(a))) - y_target

# test function
print("\n=== Function Test: Vertical Distance at 45 Degrees ===")
print(trajectory(45,x=12,g=9.8065,r=0.30,v=20))

# evaluate function
deg = [d for d in np.linspace(0,90+1,10000)]
y_dist = [trajectory(d) for d in deg]

# set plot bounds
y_lb = -10
y_ub = 15

# filter plot points
plot_points = pd.DataFrame(data=[deg,y_dist], index=["deg","y_dist"]).T
plot_points = plot_points[(plot_points["y_dist"] >= y_lb) &␣
 ↪(plot_points["y_dist"] <= y_ub)]

# plot points in bounds for visibility
plt.figure(figsize=(15,10))
plt.plot(plot_points["deg"],plot_points["y_dist"], label=r"$y(\alpha)$")

# axes, grid, labels, title
plt.axhline(y=0, color="black")
plt.axhline(y=3, color="blue", linestyle="--",label="y=3")
plt.axvline(x=0, color="black")
plt.grid()
plt.legend()
plt.xlabel(r"$\alpha$ (deg.)")
plt.ylabel("y")
plt.title("Vertical Distance:"+r" $y(\alpha)$")
plt.show()
```

```
=== Function Test: Vertical Distance at 45 Degrees ===
7.726534477443096
```



a.

I use the bisection method to find the two roots of $g(\alpha) = 0$ between 0 and 90 degrees, using the graph above to select the intervals. The graph shows one apparent root between 20 and 30, and another between 70 and 75. The algorithm works by finding a very short interval $[a, b]$ over which the function g has a change in sign. Note that g is continuous and its domain contains the interval. The sign changes indicate that $g(a) < 0 < g(b)$, thus by the intermediate value theorem, there must exist a root r somewhere in the interval such that $g(r) = 0$. The method begins with an initial interval and iteratively reduces its length by evaluating the function at the midpoint of that interval, and setting the left or right interval bound to the midpoint depending on the sign of the function at the midpoint. The algorithm stops when the final interval is smaller than some tolerance. Per Heath, an error tolerance of ϵ requires

$$\lceil \log_2 \left(\frac{b-a}{\epsilon} \right) \rceil$$

iterations. I stop when the interval length is $< 10^{-6}$ degrees.

[3]:
```
def bisection_method(a,b,tol,f,max_iter=10000, **kwargs):
    """
```

```python
    Implementation of bisection method for finding root of
    continuous nonlinear function in 1 dimension.
    Args:
    -----
        a (float): left of interval
        b (float): right of interval
        tol (float): error tolerance for length of final interval
        f (function): function to evaluate
        max_iter (int): maximum iterations of while loop
    Returns:
    --------
        m (float): midpoint of final interval
        history (array): iterate, function value, interval
    Implementation follows Heath 2E, pp.224, with early
    stopping criteria for while loop.
    """
    # initialize counter
    it = 0

    # compute required iteration
    req_it = math.ceil(np.log2((b-a)/tol))

    # track algorithms history
    a_hist, b_hist, m_hist, it_hist, fa_hist, fb_hist = [], [], [], [], [], []

    # loop until interval smaller than tolerance
    while b-a > tol:

        # compute midpoint of interval
        m = a + (b-a)/2

        # evaluate function at a and m
        f_a = f(a, **kwargs)
        f_m = f(m, **kwargs)
        f_b = f(b, **kwargs)

        # update trackers
        a_hist.append(a)
        b_hist.append(b)
        m_hist.append(m)
        it_hist.append(it)
        fa_hist.append(f_a)
        fb_hist.append(f_b)

        # check if function values have same sign
        if (f_a >= 0 and f_m >= 0) or (f_a < 0 and f_m < 0):
```

```
            # set midpoint as left bound of interval
            a = m

        # if not, compress from the right
        else:

            # set midpoint as right bound of interval
            b = m

        # increment counter
        it +=1

        # break if maximum iterations exceeded
        if it > max_iter:
            print("Warning: maximum iterations exceeded.")
            break

        # create dataframe with history
        hist = pd.DataFrame({f"i (<={req_it})":it_hist, "a":a_hist, "f(a)":
 ↪fa_hist, "b":b_hist, "f(b)":fb_hist, "m":m_hist, })


    # return root and table of iterates
    return m, hist

# call function using graph for initial values
r1, r1_hist = bisection_method(20,30,1e-6,trajectory, y_target=3)
r2, r2_hist = bisection_method(70,75,1e-6,trajectory, y_target=3)

# evaluate roots
eval_r1 = trajectory(r1, y_target=3.0)
eval_r2 = trajectory(r2, y_target=3.0)

# compute error, check within 1e-5
e1 = np.abs(eval_r1)
e2 = np.abs(eval_r2)
assert( e1 < 1e-5 and e2 < 1e-5)

print("=== Bisection Method Results ===")
print(f"Root 1: {r1}, g({r1})={eval_r1}, error={e1}")
print(f"Root 2: {r2}, g({r2})={eval_r2}, error={e2}")

# check results
r_hist = pd.concat([r1_hist, r2_hist], axis=1, keys=['Root 1 Function History',␣
 ↪'Root 2 Function History'])
display(r_hist)
```

```
=== Bisection Method Results ===
Root 1: 24.470565915107727, g(24.470565915107727)=8.635240078547213e-09,
error=8.635240078547213e-09
Root 2: 72.75664985179901, g(72.75664985179901)=-1.6550040271567923e-06,
error=1.6550040271567923e-06
```

Root 1 Function History \

	i (<=24)	a	f(a)	b	f(b)
0	0	20.000000	-9.300113e-01	30.000000	1.187451e+00
1	1	20.000000	-9.300113e-01	25.000000	1.116979e-01
2	2	22.500000	-4.126334e-01	25.000000	1.116979e-01
3	3	23.750000	-1.514349e-01	25.000000	1.116979e-01
4	4	24.375000	-2.012254e-02	25.000000	1.116979e-01
5	5	24.375000	-2.012254e-02	24.687500	4.572263e-02
6	6	24.375000	-2.012254e-02	24.531250	1.278398e-02
7	7	24.453125	-3.673276e-03	24.531250	1.278398e-02
8	8	24.453125	-3.673276e-03	24.492188	4.554350e-03
9	9	24.453125	-3.673276e-03	24.472656	4.402872e-04
10	10	24.462891	-1.616557e-03	24.472656	4.402872e-04
11	11	24.467773	-5.881504e-04	24.472656	4.402872e-04
12	12	24.470215	-7.393550e-05	24.472656	4.402872e-04
13	13	24.470215	-7.393550e-05	24.471436	1.831749e-04
14	14	24.470215	-7.393550e-05	24.470825	5.461943e-05
15	15	24.470520	-9.658095e-06	24.470825	5.461943e-05
16	16	24.470520	-9.658095e-06	24.470673	2.248065e-05
17	17	24.470520	-9.658095e-06	24.470596	6.411276e-06
18	18	24.470558	-1.623410e-06	24.470596	6.411276e-06
19	19	24.470558	-1.623410e-06	24.470577	2.393933e-06
20	20	24.470558	-1.623410e-06	24.470568	3.852611e-07
21	21	24.470563	-6.190746e-07	24.470568	3.852611e-07
22	22	24.470565	-1.169067e-07	24.470568	3.852611e-07
23	23	24.470565	-1.169067e-07	24.470567	1.341772e-07

Root 2 Function History \

	m	i (<=23)	a	f(a)	b
0	25.000000	0.0	70.000000	5.904123e+00	75.000000
1	22.500000	1.0	72.500000	8.252469e-01	75.000000
2	23.750000	2.0	72.500000	8.252469e-01	73.750000
3	24.375000	3.0	72.500000	8.252469e-01	73.125000
4	24.687500	4.0	72.500000	8.252469e-01	72.812500
5	24.531250	5.0	72.656250	3.320499e-01	72.812500
6	24.453125	6.0	72.734375	7.472324e-02	72.812500
7	24.492188	7.0	72.734375	7.472324e-02	72.773438
8	24.472656	8.0	72.753906	9.235174e-03	72.773438
9	24.462891	9.0	72.753906	9.235174e-03	72.763672
10	24.467773	10.0	72.753906	9.235174e-03	72.758789
11	24.470215	11.0	72.756348	1.016195e-03	72.758789
12	24.471436	12.0	72.756348	1.016195e-03	72.757568

13	24.470825	13.0	72.756348	1.016195e-03	72.756958
14	24.470520	14.0	72.756348	1.016195e-03	72.756653
15	24.470673	15.0	72.756500	5.022653e-04	72.756653
16	24.470596	16.0	72.756577	2.452895e-04	72.756653
17	24.470558	17.0	72.756615	1.167989e-04	72.756653
18	24.470577	18.0	72.756634	5.255289e-05	72.756653
19	24.470568	19.0	72.756643	2.042973e-05	72.756653
20	24.470563	20.0	72.756648	4.368111e-06	72.756653
21	24.470565	21.0	72.756648	4.368111e-06	72.756650
22	24.470567	22.0	72.756649	3.527015e-07	72.756650
23	24.470566	NaN	NaN	NaN	NaN

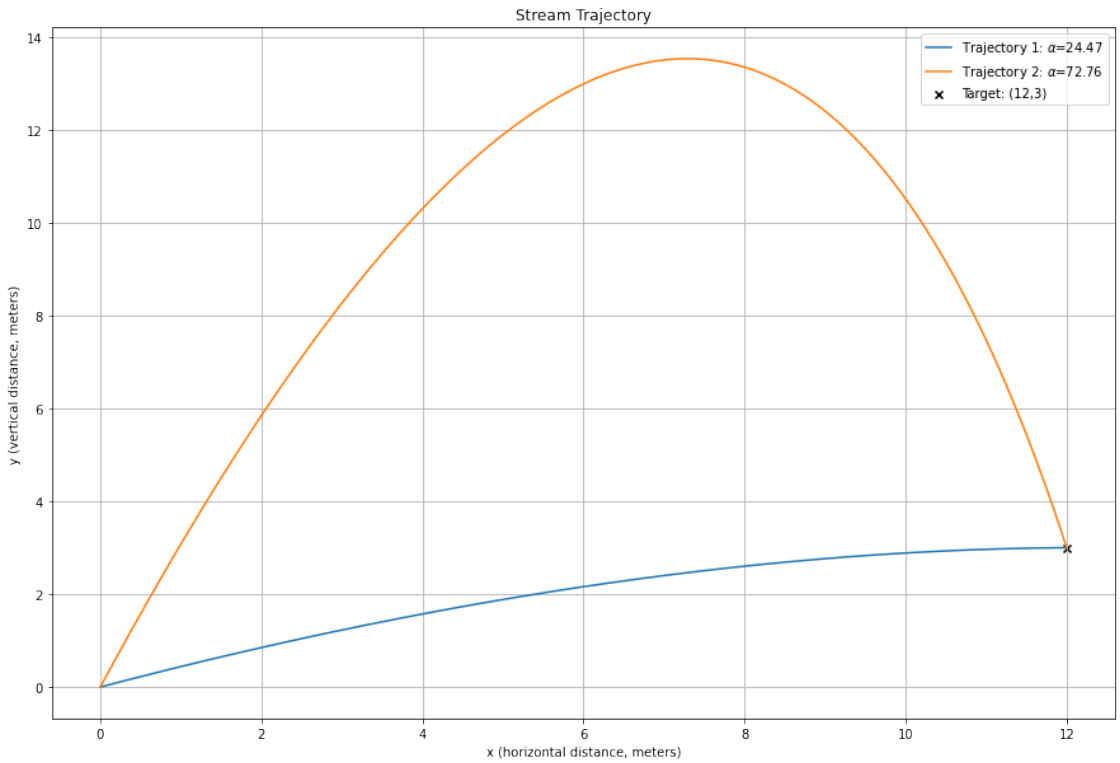
	f(b)	m
0	-11.988559	72.500000
1	-11.988559	73.750000
2	-4.049020	73.125000
3	-1.328838	72.812500
4	-0.190064	72.656250
5	-0.190064	72.734375
6	-0.190064	72.773438
7	-0.056723	72.753906
8	-0.056723	72.763672
9	-0.023685	72.758789
10	-0.007210	72.756348
11	-0.007210	72.757568
12	-0.003096	72.756958
13	-0.001040	72.756653
14	-0.000012	72.756500
15	-0.000012	72.756577
16	-0.000012	72.756615
17	-0.000012	72.756634
18	-0.000012	72.756643
19	-0.000012	72.756648
20	-0.000012	72.756650
21	-0.000004	72.756649
22	-0.000004	72.756650
23	NaN	NaN

The optimal angles are $\boxed{\alpha^* = 24.470565915107727^\circ}$ and $\boxed{\alpha^{**} = 72.75664985179901^\circ}$.

b

Using the computed roots from 1(a), I plot the trajectory of the stream from $x = 0$ to $x = 12$. Again, I obtained the computed roots using the bisection method and a graph of the function for initial values.

[4]:
```
# empty lists to hold computed trajectories
traj_1, traj_2 = [], []

# evaluate from x=0 to 12, inclusive
for x in np.linspace(0, 12, 130):

    # compute trajectory 1 and 2
    traj_1.append(trajectory(a=r1,x=x,g=9.8065,r=0.30,v=20,y_target=0))
    traj_2.append(trajectory(a=r2,x=x,g=9.8065,r=0.30,v=20,y_target=0))

# plot trajectories
plt.figure(figsize=(15,10))
plt.plot(np.linspace(0, 12, 130), traj_1, label="Trajectory 1:␣
 ↪"+rf"$\alpha$={np.round(r1,2)}")
plt.plot(np.linspace(0, 12, 130), traj_2, label="Trajectory 2:␣
 ↪"+rf"$\alpha$={np.round(r2,2)}")

#scatter x and y target
plt.scatter(x=12, y=3, label="Target: (12,3)", marker="x", color="black")

# grid, title, labels, legend
plt.grid()
plt.title("Stream Trajectory")
plt.xlabel("x (horizontal distance, meters)")
plt.ylabel("y (vertical distance, meters)")
plt.legend()
plt.show()
```



Q2

I solve the differential equation

$$u''(t) + e^{u(t)} = 0$$

for $0 < t < 1$, such that $u(0) = u(1) = 0$ by dividing the unit interval into n equal length sub-intervals and placing knots at

$$t_0 = 0, t_1 = h, t_2 = 2h, \dots, t_i = ih, t_n = nh = 1,$$

for h the length of each sub-interval. For example, $h = 0.10$ implies $n = 10$ sub-intervals. Let $\mathbf{u} = (u_1, \dots u_{n-1})^\top \in \mathbb{R}^{n-1}$, where $u_0 = u_n = 0$, be approximate values of the function $u(t)$ at knot t. Discretizing the second derivative at each knot gives a system of finite difference equations $\mathbf{f}(\mathbf{u}) = \mathbf{0}$, which has $n-1$ equations in $n-1$ unknowns,

$$\begin{bmatrix} \frac{1}{h^2}(u_2 - 2u_1 + u_0) + e^{u_1} \\ \frac{1}{h^2}(u_3 - 2u_2 + u_1) + e^{u_2} \\ \vdots \\ \frac{1}{h^2}(u_n - 2u_{n-1} + u_{n-2}) + e^{u_{n-1}} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

I solve the system using Newton's method. Because $\mathbf{f} : \mathbb{R}^{n-1} \to \mathbb{R}^{n-1}$, the Jacobian is an $n-1 \times n-1$ matrix with the i, j^{th} element given by

$$J_{ij} = \begin{cases} -\frac{2}{h^2} + e^{u_i} & \text{if } i = j \text{ for } i = 1, \dots, n-1 \\ \frac{1}{h^2} & \text{if } i+1 = j \text{ for } i = 1, \dots, n-2 \\ \frac{1}{h^2} & \text{if } i-1 = j \text{ for } i = 2, \dots, n-1 \\ 0 & \text{otherwise.} \end{cases}$$

Following Heath 2E (pp.239), the procedure is as follows.

For an initial guess

$$\mathbf{u}^{[0]} = \alpha(t_1(1-t_1), t_2(1-t_2), \dots, t_{n-1}(1-t_{n-1}))^\top,$$

iterate for $k = 0, 1, 2, \dots$

$$\begin{aligned} \text{Solve } \mathbf{J}_f(\mathbf{x}_k)\mathbf{s}_k &= -\mathbf{f}(\mathbf{x_k}) \\ \mathbf{x}_{k+1} &= \mathbf{x}_k + \mathbf{s}_k \end{aligned}$$

and stop when the L2 norm $|| \mathbf{f}(\mathbf{u}^{[k]}) ||<$ tol or the maximum number of iterations is exceeded. I repeat the exercise at $\alpha = 0, 10, 20, 50$ with $h = 0.10$ and $\alpha = 20$ with $h = 0.04$ and $n = 25$.

[5]:
```
def u_0(alpha, h):
    """
    Construct initial guess for u,
    u0 = alpha(t1(1-t1), t2(1-t2),...,tn-1(1-tn-1))
    Args:
    ----
        alpha (float): multiplier
        h (float): length of subintervals
    Returns:
    --------
        u (array): initial guess
    Divide unit interval into n equal-length subintervals,
    placing knots at t1=h...tn-1 = (n-1)h, thus u is n-1.
    """
    # compute number of knots
    n = int(1/h)

    # initialize u
    u = []

    # populate u
    for i in range(1,n):

        # compute ti
```

```
        t_i = i*h

        # set ith element to ti*(1-ti)
        u.append(t_i*(1-t_i))

    # scale by alpha
    u = alpha*np.array(u)

    # check dimension
    assert(len(u)==n-1)

    # return u
    return u


def finite_difference_system(u, h, u0=0, un=0):
    """
    Construct vector of finite difference equations f(u),
    where f: R^n-1 -> R^n-1.
    Args:
    -----
        u (array): n-1 x 1 vector
        h (float): subinterval length
        u0 (float): boundary condition
        un (float): boundary condition
    Returns:
    --------
        f (array): n-1 x 1 vector of function values
    Use dictionary to separate zero-based array indexing for
    assignment from 1 baased labeling in the setup.
    """
    # capture input length of u
    n_1 = len(u)

    # populate dictionary for u
    u = {f"u{i+1}":u[i] for i in range(0,len(u))}

    # add u0 and un
    u[f"u{n_1+1}"] = un
    u["u0"] = u0


    # initialize f
    f = []

    # loop through function indices 1...n-1 (+1 to include end of range)
    for i in range(1,n_1+1):
```

```
        # f_i = (1/h^2)(u_i+1 - 2u_i + u_i-1) + e^u_i
        f.append((u[f"u{i+1}"] - 2*u[f"u{i}"] + u[f"u{i-1}"])/(h**2) + np.
↪exp(u[f"u{i}"]) )

    # return f as an array
    return np.array(f).reshape((len(f),1))

def Jacobian(u, h):
    """
    Construct Jacobian for system of finite difference equations.
    Args:
    -----
        u (array): n-1 x 1 vector of approximate function values at knots
        h (float): length of subinterval
    Returns:
    --------
        J (array): n-1 x n-1 Jacobian matrix.
    """

    # u is length n-1
    n_1 = len(u)

    # construct zero array
    J = np.zeros(shape=(len(u),len(u)))

    # loop through row indices
    for i in range(0, n_1):

        # loop through column indices
        for j in range(0,n_1):

            # populate diagonal element
            if i == j:

                # df_i/du_i = -2/h^2 + e^u_i
                J[i][j] = -2/(h**2) + np.exp(u[i])

            # populate lower triangular elements
            if i+1==j:

                # df_i/du_i+1 = 1/h^2
                J[i][j] = 1/(h**2)

            # populate upper triangular elements
            if i-1 == j:
```

```
                # df_i/du_i-1 = 1/h^2
                J[i][j] = 1/(h**2)

    # return Jacobian
    return J


def newton_system(x0, func, Jacob, h, tol=10**-9, maxit=10):
    """
    Solve system of nonlinear equations using Newton's method.
    Code follows Algorithm 5.4 in Heath 2E, pp.239.
    Args:
    -----
        x0 (array): initial guess
        func (function): function for nonlinear system
        Jacob (function): function for Jacobian of system
        h (float): length of subinterval
        tol (float): tolerance for zero
        maxit (int): maximum number of iterations
    Returns:
    --------
        x_k (array): solution to system
        history (pd.DataFrame): norm values for iterates
    Stop early if L2 norm of function < tolerance. Otherwise,
    stop when maximum iterations reached. Track norm history.
    """
    # empty lists for iterates and norms
    history = []

    # reshape initial guess to n-1 x 1
    x_k = x0.reshape((len(x0),1))

    # check norm at guess
    l2_norm = np.linalg.norm(func(x_k, h), 2)

    # track norm value per iterate
    history.append([0,l2_norm])

    # iterate until maximum iteration count reached
    for k in range(1, maxit):

        # compute Jacobian
        J_f = Jacob(x_k, h)

        # evaluate function
        f_x_k = func(x_k, h)
```

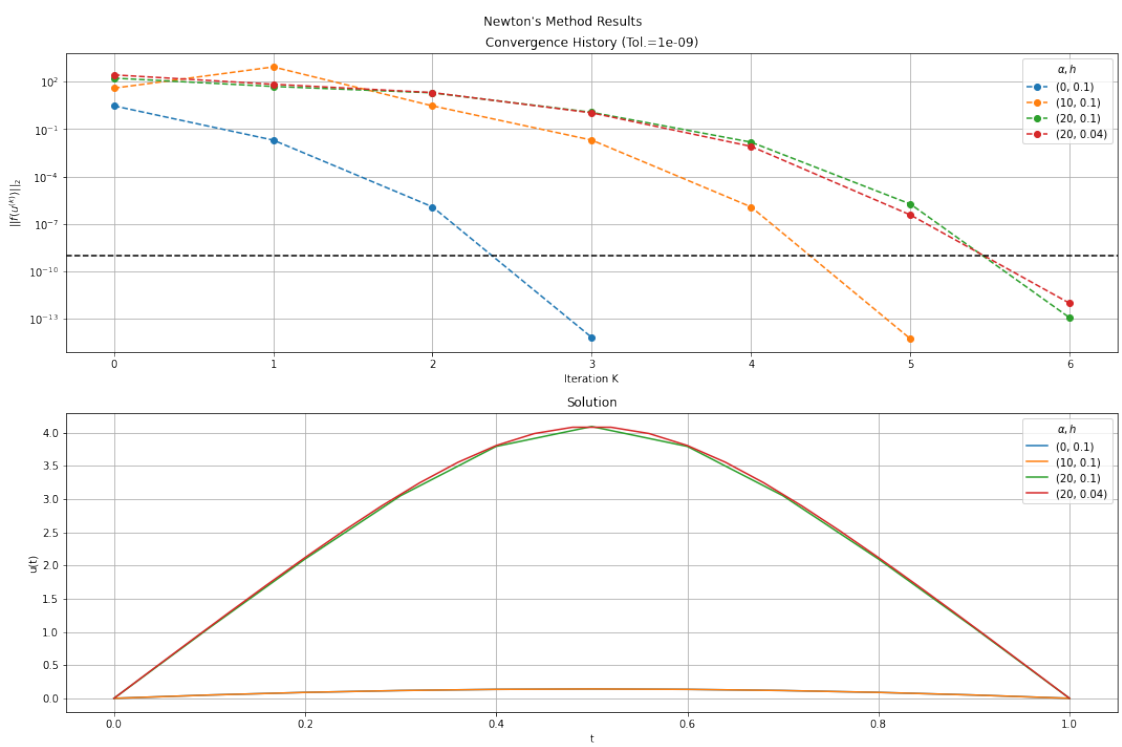
```
        # solve system
        s_k = np.linalg.solve(J_f, -f_x_k)

        # update x_k
        x_k = x_k + s_k

        # check norm
        l2_norm = np.linalg.norm(func(x_k, h), 2)

        # track norm value per iterate
        history.append([k,l2_norm])

        # stop if norm is within tolerance of 0
        if l2_norm < tol:
            break

    # return solution and history as dataframe
    history = pd.DataFrame(history, columns=["K","L2"])
    return x_k, history

# set grid for alpha and h
param_grid = [(0,0.10),(10,0.10),(20,0.10),(50,0.10),(20,0.04)]
tol = 10**-9

# subplots for convergence history and solution
fig, axs = plt.subplots(2,1, figsize=(15,10))

# solve system at each parameter combination
for params in param_grid:

    # capture params
    alpha, h = params[0], params[1]

    # compute u0
    u0_init = u_0(alpha, h)

    # solve system
    soln, hist = newton_system(u0_init, finite_difference_system, Jacobian,␣
 ↪h=h, tol=tol, maxit=10)

    # check results
    print(f"\n===> Alpha: {alpha}, h: {h} <===")
    print("\nSolution:\n",soln.T)
    print("\nConvergence history:")
    display(hist)

    # check algorithm converged converged
```

```
    if hist["L2"].values[-1] < tol:

        # plot convergence history
        axs[0].plot(hist["K"], hist["L2"], marker="o", linestyle="--",␣
 ↪label=params)

        # plot solution with u0=un=0 appended
        plot_soln = [0] + list(soln.flatten()) + [0]
        axs[1].plot([i*h for i in range(0,int(1/h)+1) ],plot_soln ,␣
 ↪label=params)

# log scale convergence plot and polish
axs[0].set_yscale('log')
axs[0].set_xlabel("Iteration K")
axs[0].set_ylabel(r"$||f(u^{(k)})||_{2}$")
axs[0].axhline(tol, linestyle="--", color="black")
axs[0].grid()
axs[0].legend(title=r"$\alpha, h$")
axs[0].set_title(f"Convergence History (Tol.={tol})")

# polish solution plot
axs[1].set_xlabel("t")
axs[1].set_ylabel("u(t)")
axs[1].grid()
axs[1].legend(title=r"$\alpha, h$")
axs[1].set_title(f"Solution")

# set super title and display plots
plt.suptitle("Newton's Method Results")
plt.tight_layout()
plt.show()
```

```
===> Alpha: 0, h: 0.1 <===

Solution:
 [[0.04989463 0.08927767 0.11772686 0.13492668 0.14068197 0.13492668
  0.11772686 0.08927767 0.04989463]]

Convergence history:

   K            L2
0  0  3.000000e+00
1  1  2.042967e-02
2  2  1.209739e-06
3  3  6.594388e-15
```

```
===> Alpha: 10, h: 0.1 <===

Solution:
 [[0.04989463 0.08927767 0.11772686 0.13492668 0.14068197 0.13492668
  0.11772686 0.08927767 0.04989463]]

Convergence history:

   K            L2
0  0  3.962969e+01
1  1  8.750015e+02
2  2  2.999822e+00
3  3  2.042880e-02
4  4  1.209634e-06
5  5  5.466087e-15


===> Alpha: 20, h: 0.1 <===

Solution:
 [[1.06290478 2.09686189 3.04941316 3.79093487 4.08947864 3.79093487
  3.04941316 2.09686189 1.06290478]]

Convergence history:

   K            L2
0  0  1.710245e+02
1  1  5.088066e+01
2  2  2.005436e+01
3  3  1.170884e+00
4  4  1.515804e-02
5  5  1.845298e-06
6  6  1.179439e-13


===> Alpha: 50, h: 0.1 <===

Solution:
 [[ 2.81059751  5.48154513  8.87245692 12.16305651 12.79639872 12.16305651
   8.87245692  5.48154513  2.81059751]]

Convergence history:

   K            L2
0  0  3.570772e+05
1  1  1.313122e+05
2  2  4.825891e+04
3  3  1.770584e+04
4  4  6.529615e+03
5  5  2.371153e+03
```

```
6  6  8.447453e+02
7  7  1.994952e+06
8  8  1.226393e+06
9  9  4.510785e+05


===> Alpha: 20, h: 0.04 <===

Solution:
 [[0.43201404 0.86156352 1.28732601 1.70729158 2.11843473 2.51626891
  2.89429139 3.24340076 3.55151701 3.80384739 3.98438017 4.07890981
  4.07890981 3.98438017 3.80384739 3.55151701 3.24340076 2.89429139
  2.51626891 2.11843473 1.70729158 1.28732601 0.86156352 0.43201404]]

Convergence history:

   K            L2
0  0  2.745050e+02
1  1  6.997406e+01
2  2  2.081081e+01
3  3  1.081268e+00
4  4  8.203735e-03
5  5  3.775332e-07
6  6  9.703554e-13
```



I plot $u(t)$ for each α that converges above. Note the following.

First, for the discretized nonlinear system, $\boxed{\text{Newton's method converges to two different solutions}}$. One occurs using $(0, 0.1)$ and $(10, 0.1)$, the other using $(20, 0.1)$ and $(20, 0.04)$. The curves peak at 0.14068197 and 4.07890981, respectively, and because Newton's method is local, the choice of initial guess determines which solution the method identifies.

Second, the large $\alpha = 50$ initial guess fails to converge within 10 iterations. $\boxed{\text{All others converge}}$ at most within 6 iterations, with $(0, 0.1)$ exhibiting the fastest convergence within 3 iterations. For parameter configurations that do converge, $\boxed{\text{convergence appears quadratic}}$, as the L2 norm drops like it has been squared at each step (e.g. $10^{-2} \to 10^{-6} \to 10^{-15}$).